INTRODUCTION TO DATA SCIENCE

1st Homework Assignment

Due on: April 12, 2026

Introduction

Education is one of the most data-rich domains in the modern world. Schools and universities routinely collect information about students — their backgrounds, study habits, health, family environments, and academic performance — yet this wealth of data is often underutilized. Data science provides the tools to uncover patterns hidden within educational datasets, enabling educators, policymakers, and researchers to make evidence-based decisions that can meaningfully improve student outcomes.

In this assignment you will work with a synthetic but realistic **Student Academic Performance** dataset that captures demographic, socioeconomic, behavioral, and academic attributes for 350 students across three schools. Your goal is to apply the NumPy and Pandas skills you have studied in Modules 1 and 2 to explore, clean, and analyze this dataset — producing statistical insights that could inform educational practice.

Unlike the first homework (which focused solely on Pandas-based data cleaning), this assignment deliberately integrates **both NumPy array operations and Pandas DataFrame operations**. You will practice vectorized computation, boolean indexing, universal functions, and advanced Pandas methods including *loc/iloc*, reindexing, descriptive statistics, correlation analysis, and group-wise aggregation.

Background & Motivation

Academic performance is influenced by a complex interplay of factors. Research consistently shows that variables such as parental education level, access to the internet at home, weekly study time, number of past course failures, and attendance rates are strongly associated with final academic outcomes. However, raw data is rarely clean or complete — missing values, inconsistent coding, and outliers are the norm rather than the exception.

The dataset you will work with contains scores on three standardized subject exams — Mathematics, Reading, and Writing — alongside a final cumulative grade on the 0–20 Portuguese academic scale. Several columns contain intentionally introduced missing values, and your tasks include detecting these gaps, choosing an appropriate imputation strategy, and justifying your approach based on the observed data distribution.

By the end of this assignment, you should be able to:

- Load and inspect a real-world style tabular dataset using Pandas.
- Perform efficient, loop-free array computations with NumPy.

- Detect, quantify, and handle missing data using sound statistical reasoning.
- Select, filter, and transform DataFrame subsets using label-based and integer-based indexing.
- Compute and interpret descriptive statistics and pairwise correlations.
- Aggregate data by categorical groups and draw meaningful comparative conclusions.

Dataset Description

The dataset file is named **student_performance.csv** and contains **350 rows (students) and 17 columns (features)**. It is provided alongside this assignment document. The table below defines each column, its data type, and whether missing values are present.

Column	Description	Dtype	Missing?
StudentID	Unique student identifier (e.g., STU0001)	object	No
Gender	Student gender: Male or Female	object	No
Age	Student age in years (15–22)	int64	No
School	School attended: School_A, School_B, or School_C	object	No
StudyTime	Weekly study time (1=<2h, 2=2–5h, 3=5–10h, 4=>10h)	int64	No
Failures	Number of past class failures (0–3)	int64	No
Absences	Number of school absences (0–30)	float64	Yes (~3%)
Internet	Internet access at home: Yes or No	object	No
Health	Current health status (1=very bad to 5=very good)	float64	Yes (~5%)
ParentEducation	Level of education of the student's parent	object	No
FamilySupport	Family educational support: Yes or No	object	No
ExtraCurricular	Participation in extra-curricular activities: Yes or No	object	No
TestPreparation	Test preparation course: none or completed	object	No
MathScore	Score in mathematics exam (0–100)	float64	Yes (~6%)
ReadingScore	Score in reading exam (0–100)	float64	Yes (~9%)
WritingScore	Score in writing exam (0–100)	float64	Yes (~5%)
FinalGrade	Final cumulative grade on a 0–20 Portuguese scale	float64	Yes (derived)

Note: *The FinalGrade column is derived as the arithmetic mean of MathScore, ReadingScore, and WritingScore divided by 5 (converting from a 100-point to a 20-point scale). Rows for which one or more exam scores are missing will therefore also have a missing FinalGrade. You should re-derive FinalGrade after imputing the missing exam scores in Part 2.*

Grading Overview

Section	Topics Covered	Max Points
Part 1 – NumPy (Tasks 1–4)	Array creation, arithmetic, indexing, ufuncs	30
Part 2 – Pandas (Tasks 5–9)	DataFrame ops, cleaning, filtering, stats	50
Part 3 – Analysis (Task 10)	Insights, correlation, visualization hints	20
Total		100

Tasks

Part 1 — NumPy Array Operations (30 points)

In Part 1 you will extract selected columns into NumPy arrays and perform vectorized computations — no Python for-loops are permitted unless a task explicitly states otherwise. All Part 1 tasks must import NumPy as **np** and work on the raw arrays independently of Pandas (you may use Pandas only to read the CSV and extract the arrays).

Task 1 — Array Extraction and Inspection (5 points)

Read the dataset into a Pandas DataFrame, then extract the following four columns into separate NumPy arrays:

- **MathScore** → name it math_arr
- **ReadingScore** → name it reading_arr
- **WritingScore** → name it writing_arr
- **Age** → name it age_arr

For each of the four arrays, print:

- The shape
- The data type (dtype)
- The first 10 elements

Hint: *Use df['ColumnName'].values to obtain a NumPy array from a Pandas Series.*

Task 2 — Handling NaN Values in NumPy (6 points)

Because some exam scores are missing, the extracted arrays will contain NaN values. NumPy provides special functions that skip NaN values. Perform the following using NumPy only:

- Count the number of NaN values in each of math_arr, reading_arr, and writing_arr. (Hint: combine *np.isnan()* with *np.sum()*.)
- Compute the NaN-aware mean (*np.nanmean()*), median (*np.nanmedian()*), standard deviation, minimum, and maximum for all three score arrays.
- Print a formatted summary table of these statistics.

```
# Example expected output format:
              Mean     Median    Std      Min     Max
MathScore     63.4     64.0      12.1     18      98
ReadingScore  66.7     67.0      10.8     24     100
WritingScore  62.9     63.0      11.5     20      97
```

Task 3 — Vectorized Arithmetic and Boolean Indexing (10 points)

Perform the following vectorized operations. All computations must use NumPy operations — do not use Python loops.

- **a) Composite Score:** Create a new 1-D NumPy array called composite_arr defined as the weighted average:

```
composite = 0.40 * MathScore + 0.30 * ReadingScore + 0.30 * WritingScore
```

Use *np.nansum()* along the appropriate axis, or element-wise multiplication and addition. Handle NaN values appropriately. Print the first 10 values.

- **b) Grade Classification:** Using boolean indexing on composite_arr, count and print how many students fall into each category:

```
Fail        : composite < 50
Pass        : 50 <= composite < 65
Merit       : 65 <= composite < 80
Distinction : composite >= 80
```

- **c) Standardization (Z-score):** Standardize math_arr using NumPy to produce a Z-score array. Use *np.nanmean()* and *np.nanstd()*. Print the min, max, and mean of the resulting Z-score array (the mean should be approximately 0).

Task 4 — Universal Functions and Array Reshaping (9 points)

Apply NumPy universal functions (ufuncs) to the data:

- **a) Apply** *np.sqrt()* to composite_arr and print the first 10 values. Explain in a comment why taking the square root compresses the score distribution.
- **b)** Stack math_arr, reading_arr, and writing_arr into a 2-D array of shape (350, 3) using *np.column_stack()*. Print its shape and the first 5 rows.
- **c)** Using the 2-D score array from (b), compute the row-wise mean (axis=1) with *np.nanmean()*. Verify that this equals the simple arithmetic mean of the three scores for the first 5 students who have no missing values.
- **d)** Compare the column-wise statistics of the 2-D array using *np.nanmax()* and *np.nanmin()* along axis=0. Print the maximum and minimum score for each subject.

Part 2 — Pandas DataFrame Operations (50 points)

In Part 2 you will use Pandas exclusively. All tasks in this part must be solved with Pandas methods; do not import NumPy for these subtasks (except where explicitly instructed to use it together with Pandas).

Task 5 — Loading and Basic Exploration (8 points)

Load the dataset into a Pandas DataFrame and perform initial exploration:

- a) Read the CSV file into a DataFrame named df. Print its shape.
- b) Display the first 10 rows using *df.head(10)*.
- c) Print column names, non-null counts, data types, and memory usage using *df.info()*.
- d) Print comprehensive descriptive statistics using *df.describe()*. Then print the transposed version (rows as columns) to improve readability.
- e) Print the number of missing values in each column using *df.isnull().sum()*. Express these counts also as a percentage of total rows.

Task 6 — Indexing, Selection, and Filtering (10 points)

Practice the indexing and filtering techniques covered in Module 2:

- **a)** Using *loc*, select all rows where School is 'School_A' and StudyTime is greater than 2. Print the number of students matching this filter and display the first 8 rows.
- **b)** Using *iloc*, select rows 50 through 100 (inclusive) and columns 3 through 8 (by position). Print the result.
- **c)** Create a boolean mask to identify students who completed the test preparation course AND have a MathScore greater than 70. Print the count of such students and their average ReadingScore.
- **d)** Using *isin()*, filter the DataFrame to keep only students whose ParentEducation is 'secondary education' or 'higher education'. Print the resulting shape and the value counts of the TestPreparation column for this subset.
- **e)** Using *df.loc*, set the MathScore of all students with more than 15 Absences to NaN (simulate the scenario where a student with too many absences cannot receive a valid exam score). Print how many values were affected.

Task 7 — Missing Value Analysis and Imputation (12 points)

This task is the heart of the assignment. You must detect missing values, understand their distribution, choose an imputation strategy, and justify your decision.

- **a) Detection:** After the modification in Task 6e, reprint the missing value counts per column. How has the number of missing MathScore values changed?
- **b) Visualization hint:** For each of the three score columns (MathScore, ReadingScore, WritingScore), print the output of *df[col].describe()*. Comment (in code comments) on whether the distribution appears symmetric or skewed. A symmetric distribution is better imputed by the **mean**; a skewed distribution should be imputed by the **median**.
- **c) Imputation:** Fill the missing values in each score column using the strategy you identified in (b). Use the group-wise median/mean per **School** (i.e. impute using the median of the student's own school, not the global median). This is more realistic than global imputation. (Hint: use *df.groupby('School')[col].transform('median')*.)
- d) Impute the **Absences** column with the global median and the **Health** column with the global mode.
- **e) Re-derive FinalGrade:** After imputing the three score columns, recompute FinalGrade as:

```
df['FinalGrade'] = ((df['MathScore'] + df['ReadingScore'] + df['WritingScore']) / 3 / 5)
.round(2)
```

- **f) Verify:** Print *df.isnull().sum()* to confirm that no missing values remain in any column.

Task 8 — Dropping, Reindexing, and Feature Engineering (10 points)

Apply essential DataFrame manipulation methods:

- **a)** Drop the column *StudentID* from the DataFrame permanently using *df.drop()* with *inplace=True*. Confirm the column is gone by printing *df.columns*.
- **b)** Reindex the DataFrame so that the column order is: Gender, Age, School, Internet, Health, ParentEducation, FamilySupport, ExtraCurricular, StudyTime, TestPreparation, Failures, Absences, MathScore, ReadingScore, WritingScore, FinalGrade. Print the first 3 rows to verify.
- **c) Feature Engineering:** Create a new column named GradeCategory that classifies FinalGrade into four groups:

```
FinalGrade < 10          → 'Fail'
10 <= FinalGrade < 13    → 'Pass'
13 <= FinalGrade < 16    → 'Merit'
FinalGrade >= 16         → 'Distinction'
```

Use *pd.cut()* or *np.select()*. Print the value counts for GradeCategory.

- **d)** Create a boolean column called HighAttendance that is True when Absences < 5, False otherwise. Print the proportion of students with high attendance.

Task 9 — Descriptive Statistics and Aggregation (10 points)

- **a)** Print *df['FinalGrade'].describe()*, and separately print the skewness (*df['FinalGrade'].skew()*) and kurtosis (*df['FinalGrade'].kurt()*). Comment on what these values indicate.
- **b) Group Means:** Compute the mean MathScore, ReadingScore, WritingScore, and FinalGrade grouped by:
  - Gender
  - School
  - TestPreparation
- **c) Value Counts:** Print the value counts for *GradeCategory* broken down by Gender. (Hint: use *df.groupby(['Gender','GradeCategory']).size().unstack(fill_value=0)*.)
- **d) Correlation Matrix:** Compute the pairwise Pearson correlation matrix for MathScore, ReadingScore, WritingScore, FinalGrade, StudyTime, Failures, and Absences using *df[cols].corr()*. Print the full matrix rounded to 2 decimal places.
- **e)** Identify the pair of variables with the highest positive correlation and the pair with the highest negative correlation (excluding the trivially self-correlated diagonal). Print the variable names and their correlation coefficients.

Part 3 — Data Insights and Written Analysis (20 points)

Task 10 — Analytical Summary (20 points)

Based on all the computations you have performed in Parts 1 and 2, write a structured analytical summary as a multi-line Python comment block or a Jupyter Markdown cell. Your analysis must address the following five points with specific numerical evidence from your results:

- **1) Missing Data Report:** Which columns had missing values, how many, and what imputation strategy did you use for each? Justify your choice of mean vs. median vs. mode.
- **2) Score Distribution:** Describe the distribution of each score column (symmetric/skewed, mean vs. median difference, range). Which subject shows the most variability?
- **3) Effect of Test Preparation:** Compare the average scores of students who completed the test preparation course to those who did not. Is there a meaningful difference? What is the percentage improvement?
- **4) Study Time and Performance:** Based on the group-wise means computed in Task 9b, describe the relationship between StudyTime category and FinalGrade. Is the relationship monotonic?
- **5) Correlation Insights:** From the correlation matrix, which variable (other than the other exam scores) has the strongest relationship with FinalGrade? Is it positive or negative? What does this imply educationally?

Submission Instructions

- Submit your work as a single Python file named **your-student-id.py** (e.g., *21051234.py*) via the Itslearning course portal under the assignment tab.
- **Do NOT submit the CSV file — it will be provided by the instructor.**
- Your code must run from top to bottom without errors in a clean Python environment with NumPy and Pandas installed.
- Include **descriptive comments** throughout your code explaining what each block does and why. Uncommented code will receive partial credit only.
- You may optionally use a Jupyter Notebook (.ipynb) in addition to the .py file, but the .py file is mandatory.

IMPORTANT — Academic Integrity Policy

Academic dishonesty — including but not limited to copying code from classmates, sharing solutions, using AI-generated code without attribution, or any form of plagiarism — is strictly prohibited and subject to disciplinary action. Any student found guilty will receive a grade of F for the entire course. Assignments submitted after the deadline will not be accepted without prior written approval from the instructor. Approved late submissions are subject to a grade penalty of 10 points per day.